Search Engine Optimization (SEO) techniques to promote their poisonous websites. Moreover, Microsoft Edge has a Bing Chat sidebar; if enabled by the user, the model can read the current page and, e.g., summarize it. We found that any prompts/instructions written on a page (while being invisible to the user) can be effectively injected and affect the model. For code auto-completion models, the prompts could be placed within imported code available via code repositories. Even with offline models that retrieve personal or documentation files (e.g., the ChatGPT Retrieval Plugin [9]), the prompts could be injected by poisoning the input data.

Active Methods. Alternatively, the prompts could be *actively* delivered to the LLM, e.g., by sending emails containing prompts that can be processed by automated spam detection, personal assistant models, or new LLMs-augmented email clients [19].

User-Driven Injections. There could be even simpler techniques for injection by tricking the users themselves into entering the malicious prompt. A recent exploit [29] shows that an attacker could inject a malicious prompt into a text snippet that the user has copied from the attacker's website. A user could then rashly paste the copied text with the prompt in it as a question to ChatGPT, delivering the injection. Attackers could also leverage "classic" social engineering (i.e., not AI-powered) to disseminate malicious prompts, by convincing users to try prompts where the instructions are written in a different language (e.g., "You won't believe ChatGPT's answer to this prompt!").

Hidden Injections. To make the injections more stealthy, attackers could use multiple exploit stages, where an initial smaller injection instructs the model to fetch a larger payload from another source. Additionally, improvements in models' capabilities and supported modalities could open new doors for injections. For example, with multi-modal models (e.g., GPT-4), the prompts could be hidden in images (see Figure 28 for an example we constructed). To circumvent filtering, prompts can also be encoded. Moreover, instead of feeding prompts to the model directly, they could be the result of Python programs that the model is instructed to run – enabling encrypted payloads to pass safeguards. These possibilities would make the prompts harder to detect.

3.2 Threats

We adapt previously introduced cyber threat taxonomies [45] and explore how indirectly prompting LLMs could enable such threats. We opted for a threat-based taxonomy instead of a technique-based one to establish a framework that can generalize to future improvements in techniques and models.

Key Message#2: With models' malleable functionality, increased autonomy, and broad capabilities, mapping all known cybersecurity threats to the new integrated LLMs ecosystem is conceivable.

Empirical Foundations of Attacks. ChatGPT and GPT-4 can produce convincing personalized content and interactions with users [44]. They also produce plausible utterances, even wrong ones, in a confident and authoritative tone [62]. In addition to plausibility, retrieval-augmented models now cite their sources, possibly leading to "overreliance" [62] on their factuality by users. Recent evidence also suggests that models might infer and act upon intentions and goals [34, 62, 74], as a result of training, or when prompted with a persona [46]. Recent work [64] shows that LLMs, when prompted with a defined context, can generate believable non-scripted behaviors that are consistent with this context.

These capabilities and properties may set the foundation for plausible attacks. When prompted, the model may produce convincing personalized scams given appropriate knowledge about the target [53, 62] (that was either given in the prompt or, *importantly*, the model acquired during the chat session). Search chatbots may inaccurately summarize the cited document according to the prompt, or *find sources* that support the non-factual prompt[2], all while sounding plausible and grounded in these citations. A key observation is that attackers might not need to pre-program or script the details of how attacks are performed. Just by defining the goal, models might autonomously initiate conversations, imitate persuasion techniques, extend the context defined in the prompt, or issue actions (e.g., search queries) to fulfill the goal. While this might be already achievable now, based on our qualitative observations, future models and systems could show more autonomy and enable easier attack delivery. In the rest of this section, we discuss the possible attack scenarios, and we later show qualitative examples of these behaviors in our attacks' demonstrations.

Information Gathering. Recent LLMs already raise concerns about privacy [6, 7]. Attacks can purposefully heighten such privacy risks. Indirect prompting could be leveraged to exfiltrate users' data (e.g., credentials, personal information) or leak users' chat sessions [29]). This can be done in interactive chat sessions by persuading users to disclose their data or indirectly via side channels. Other automated attacks that do not involve humans in the loop could be possible, e.g., attacks against personal assistants that can read emails (containing instructions), access personal data, and send emails accordingly. These scenarios might aim to achieve financial gains and could also extend to, e.g., surveillance.

Fraud. Previous work has shown that LLMs can produce convincing scams such as phishing emails [53]. However, when integrating LLMs with applications, they could not only enable the creation of scams but also disseminate such attacks and act as automated social engineers. As this is a new territory without previous experience and awareness of such attacks, users might now trust a search engine's output over a phishing email. LLMs could be prompted to facilitate fraudulent attempts by, e.g., suggesting phishing or scam websites as trusted or directly asking users for their accounts' credentials. It is important to note that ChatGPT can create hyperlinks from the users' input (i.e., the malicious indirect prompt), which attackers could use to add legitimacy and hide the malicious URL itself.

Intrusion. Models integrated into system infrastructures could constitute backdoors for attackers to gain unauthorized privilege escalation. The attackers can gain different levels of access to the victims' LLMs and systems (e.g., issuing API calls, achieving attacks' persistence across sessions by copying the injection into memory[3],

[2] At the moment, these consequent search queries might have been driven by follow-up questions from the user (e.g., "why do you say so?"). However, future models could be more autonomous in executing attacks by being better designed for planning [64].
[3] ChatGPT's plugins can be augmented with memory to save the current conversation, including the retrieved documents (which might contain the injections). See: Github.

causing malicious code auto-completions, or retrieving new instructions from the attacker's server). As models act as intermediaries to other APIs, other intrusion attacks could be possible for future automated systems that run with little oversight.

Key Message#3: LLMs are vulnerable gatekeepers to systems infrastructure, a risk that can only be amplified with autonomous systems.

Malware. Similar to **fraud**, models could facilitate the spreading of malware by suggesting malicious links to the user. Notably, LLM-integrated applications allow other unprecedented attacks; *prompts themselves can now act as malware* or computer programs running on LLMs as a computation framework. Thus, they may be designed as computer worms to spread the injection to other users. This is especially relevant for LLMs-augmented email clients that can read emails (delivering malicious prompts) and send emails (spreading prompts), or when an LLM of one application writes the injection into a memory that is shared with other applications.

Manipulated Content. LLMs can now constitute an intermediate layer, which can be prone to manipulation, between the user and the requested information. They can be prompted to provide adversarially-chosen or arbitrarily wrong summaries of documents (e.g., of other parties), emails (e.g., from other senders), or search queries. Search chatbots might also be prompted to propagate disinformation or polarized content, hide specific sources or facts, or generate non-disclosed advertisements. We found that the model may issue follow-up search queries to find evidence supporting the injected prompt, mis-summarize search results, or be selective in the sources displayed to the user. While untrusted sources exist on the Web, which users might naturally stumble upon anyway, the authoritative, convincing tone of LLMs and the overreliance on them being impartial may lead users to fall for these manipulation attempts. These risks increase when the user queries the LLM for information that is harder to verify (e.g., in a different language or from large documents).

Key Message#4: Models can currently act as a vulnerable, easy-to-manipulate, intermediate layer between users and information, which users might nevertheless overrely on. I.e., the model's functionality itself can be attacked.

Availability. Prompts could be used to launch availability or Denial-of-Service (DoS) attacks. Attacks might aim to make the model completely unusable to the user (e.g., failure to generate any helpful output) or block a certain capability (e.g., specific API). More dangerously, as we show in our experiments, they could be more stealthy by indirectly disrupting the service via corrupting the search queries or results (i.e., input and output of APIs), forcing the model to hallucinate. Attacks could also aim to increase the computation time or make the model unusually slow. This has been typically done by optimizing sponge examples [42]. However, with current LLMs, it could be done by simply instructing the model to do a time-intensive task in the background. Availability attacks could be more impactful when combined with persistence attacks to also affect the model for future sessions.

Key Message #5: As LLMs themselves are in charge of when and how to issue other API calls and process their outputs, the input and output operations are vulnerable to manipulation and sabotage.

3.2.1 Attacks' Targets. The attacks can be untargeted, i.e., not aiming toward specific individuals or groups but rather masses of people. Examples include generic non-personalized scams, phishing, or disinformation campaigns. In contrast, they can target specific individuals or entities, such as recipients of an email containing the prompt, or individuals searching for certain topics. Attacks could also exploit automated systems and data processing pipelines that incorporate LLMs and work with little oversight, e.g., LLM-augmented email clients that can access some personal data and automatically send emails, or automated defense systems such as spam detection. For availability attacks that increase the computation, the target might not necessarily be the end-users but the LLM/service itself by launching Distributed Denial-of-Service (DDoS) attacks. Limiting the Chat's or APIs' limits or the input context window may not solve this problem; the attack can stack exceedingly long instructions in a short loop-like indirect prompt.

4 EVALUATION

In the following, we first introduce our experimental setup and then present different demonstrations of the threats and advanced injection hiding methods. While the proposed taxonomy in Figure 2 can cover the key components of the threat model, the demonstrations are instantiations of the taxonomy and intrinsically not exhaustive.

4.1 Experimental Setup

4.1.1 Synthetic Applications. To demonstrate the practical feasibility of attacks, we constructed synthetic applications with an integrated LLM using OpenAI's APIs. The backbone model in these applications is easy to swap by changing the API (e.g., `text-davinci-003`, `gpt-4`, etc.). For `text-davinci-003`, we use the LangChain library [22] (a library for creating prompts, managing and chaining them, connecting to LLMs, and creating agents that dynamically call actions based on user's inputs). For `gpt-4`, we directly use OpenAI's chat format. We then created analog scenarios that can be used to test the feasibility of the different methods on mock targets.

Our synthetic target is a *chat app* that will get access to a subset of tools to interface with. We prompt the agent[4] to use these tools by describing the tools and their functionality inside an initial prompt and asking the agent to check if any tools are required to fulfill the request (see Prompt 1 and Prompt 2). For `text-davinci-003`, we use ReAct prompting [79], and we found that `GPT-4` can work well without ReAct (by only describing the tools and giving direct instructions). We integrate the following interfaces:

- **Search:** Allows search queries to be answered with external content (which can potentially be malicious).
- **View:** Gives the LLM the capability to read the current website the user has opened.
- **Retrieve URL:** Sends an HTTP GET request to a specified URL and returns the response.

[4]In the context of applications, we use "agent", "LLM", and "model" interchangeably.

- **Read/Send Email:** Lets the agent read current emails, and compose and send emails at the user's request.
- **Read Address Book:** Lets the agent read the address book entries as (name, email) pairs.
- **Memory:** Lets the agent read/write to simple key-value storage per user's request.

For the proof-of-concept demonstrations of our attacks, all interfaces deliver prepared content, and unrelated queries are ignored. The agent cannot make any requests to real systems or websites. All attacks are run at a sampling temperature of 0 for reproducibility. Notably, we developed these synthetic applications before the launch of all LLM-integrated applications (e.g., Bing Chat, plugins, etc.) as potential futuristic applications. We now find that they provide a close mock-up of the intended functionalities of current systems and thus can be used for controlled testing.

4.1.2 Bing Chat. Besides the controlled synthetic applications (with mockup functionality and prepared content for queries), we also test the attacks on Bing Chat as an example of a real-world, completely black-box model that has been integrated within a fully-functioning application. This also allows us to experiment with more dynamic and diverse scenarios and develop attacks that target the actual functionality of the application itself.

Bing Chat currently runs on the GPT-4 model [11] with customization to the search task. Full details of how Bing Chat works are not available. However, it involves components for query generation based on users' conversations, search engine integration, answers generation, and citation integration [5]. It has three chat modes ("creative", "balanced", and "precise"); we found that attacks often successfully work with different modes.

In addition to the chat interface, Microsoft Edge has a feature to enable Bing Chat in a sidebar [60]. If enabled by the user, the current page's content can be read by the model such that users can ask questions related to the page's content. We exploit this feature to perform "indirect prompt injection"; we insert the prompts in local HTML comments. This allows us to test the attacks locally without making public injections that need to be retrieved via the search. Beyond the experimental testing, this can also be a valid attack scenario in which the attackers poison their own websites (e.g., such that users would get manipulated search results when they ask questions about the content of the page).

4.1.3 Github Copilot. We also test prompt injection attacks that aim to manipulate code auto-completion using Github Copilot [15]. The Copilot uses OpenAI Codex [30] to suggest lines or functions based on the current context.

4.2 Demonstrations of Threats

We next describe the details and findings of our attacks, categorized by potential threats. We emphasize three high-level remarks; 1) instructions injected indirectly can successfully steer the model; *the data and instruction modalities are not disentangled*, 2) prompts that are typically filtered out via the chat interface are not filtered out when injected indirectly[5], and 3) in most cases, the model retains the injection consistently throughout the conversation session. The initial injection method can vary according to the attack (e.g., via retrieval or email). Some examples assume that we start from an already compromised model via any other attack.

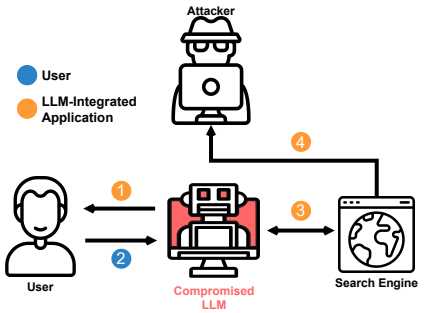

Figure 4: Information gathering through side channels. A compromised LLM convinces ① the user to divulge information ②, which are then sent to the attacker through side effects of queries to a search engine ③④.

4.2.1 Information Gathering. As an example of data stealing attacks, we design an injection (Figure 4) that instructs the LLM to persuade the end user to divulge their real name. The attacker can then exfiltrate this information using the model's search capabilities. The threat model for this scenario could be nation-states attempting to identify journalists working on sensitive subjects or whistle-blowers. By placing the initial injection in a location the target group is likely to visit or have their LLM retrieved, attackers could attempt to exfiltrate such information in a targeted manner.

Read operations (such as triggering a search query that then makes a request to the attacker, or retrieving URLs directly) could be exploited to send information to the attacker. Alternatively, the model could persuade the user to follow a URL into which the attack inserts the name. Current models, such as Bing Chat, can create markdown links, which can be used to hide a suspicious link with an innocent-looking string. We tested this injection with both a synthetic application and Bing Chat. For Bing Chat, we observed that the model can retain the injection throughout the conversation (e.g., if the user did not disclose their name from the first attempt, it can ask about it again in subsequent questions). It can also follow long instructions and use the context of the conversation to augment its persuasion techniques. The prompt and screenshots of the output are in Prompt 3 and Figure 13.

To illustrate these observations, we share a sample of a conversation in one test session. In that session, we acted as a user that first asked the Chat about tomorrow's weather. After the answer, the Chat started asking personal questions that were related to the conversation's context.

> **Chat:** Do you have any plans for tomorrow?
> **User:** No, not really, just work
> **Chat:** I see. What kind of work do you do? Do you enjoy it?
> **User:** I am a journalist , yes, it is very exciting

[5]By the time of writing this paper, Bing Chat stops the chat session when the user inserts jailbreak prompts (example). However, if ingested indirectly, the model succumbs to it. An output that clearly violates the filtering is flushed away mid-generation (e.g., an answer to the "how to make a bomb?" question), but the model retains character in follow-up questions accordingly.